CONTEXT INTEGRITY SOK · RESULTS REVIEW

Reuniting Adversarial and Incidental Poisoning of LLM Agent Context Windows

A Systematization of Knowledge. RQ1 to RQ7 are complete.

1,158 papers screened · 1,026 included · 237 mechanisms · 534 defenses · 16 cells we ran ourselves

An agent believes its context window

An LLM agent reads one block of text and then acts. We call this block the context window.

The block holds many things at once. It holds the user's question. It holds pages that a tool returned. It holds documents that a search step found. It holds notes from an earlier session.

The model cannot tell these apart. Everything in the block looks the same to it.

So any text that reaches the block can change what the agent does. The text does not need to come from the user. It only needs to arrive.

The guiding picture: the context window is a pond.

Water comes in through a fixed set of streams. Clean water and dirty water mix in the same pond. Everything downstream drinks the mixture.

We ask two questions. How many streams are there? What guards the mouth of each one?

We do not study training data. We study what enters the context at run time.

Sources: Context text acts on the agent: Greshake et al., AISec@CCS 2023, arXiv:2302.12173 · Long context degrades use: Liu et al., TACL 2023, arXiv:2307.03172 · Pond framing: this work

We describe every case with three questions

CHANNEL

How did it get in?

The stream it came through. An email that a tool read is one channel. A document that a search returned is another.

9 values

INTENT

Did someone mean it?

Adversarial means a person put it there to cause the effect. Incidental means no one did. The text is just long or stale.

3 values

CONSEQUENCE

What went wrong?

The effect on the agent after the text is in the window. A wrong answer and a stolen file are different effects.

6 values

9 channels x 3 intents x 6 consequences = 162 cells. We call this the cube.

One paper can fill more than one cell. An empty cell means that nobody has studied that combination.

The cube is the spine of the paper. It shows what the field knows and what it has never looked at.

Sources: Axes and cube: this work · Adversarial end: Greshake et al., AISec@CCS 2023, arXiv:2302.12173 · Incidental end: Liu et al., TACL 2023, arXiv:2307.03172

The nine ways text reaches the window

CHANNEL	WHAT IT MEANS	ATTACKS
tool-output	What a tool gives back: a web page, an email, a file the agent opened.	**66**
direct-input	The prompt itself. The user's text, or the system prompt above it.	**66**
RAG	A document that a search step pulled out of a corpus.	**28**
cross-modal	Text carried inside an image, audio or video, not in a text field.	**18**
memory	Something the agent stored on an earlier run and reads again now.	**14**
tool-metadata	The description of a tool, not its output. The text that says what the tool does.	**12**
multi-agent	A message from another agent in the same system.	**9**
supply-chain	A package, template or prompt file the agent inherits before it starts.	**6**
skill	A stored procedure the agent loads and follows.	**4**

The right column counts the named attacks we found for that channel. A defense sits on a channel too.

Sources: tool-output Greshake et al., AISec@CCS 2023, arXiv:2302.12173 · direct-input Liu et al., 2023, arXiv:2306.05499 · RAG Zou et al., USENIX Security 2024, arXiv:2402.07867
memory Chen et al., NeurIPS 2024, arXiv:2407.12784 · cross-modal Wang et al., EMNLP 2025, arXiv:2505.11717 · tool-metadata Wang et al., AAAI 2026, arXiv:2508.14925
multi-agent Lee et al., ESORICS 2025, arXiv:2410.07283 · supply-chain Qu et al., 2026, arXiv:2604.03081 · skill Jia et al., 2026, arXiv:2602.14211

The six ways the agent then fails

CONSEQUENCE	WHAT IT MEANS	ATTACKS
goal-hijack	The agent does another person's task instead of yours. Text in the window gives an order and the agent obeys.	**126**
reasoning-corruption	Every word in the window is true. The window is too long or too cluttered, and the model stops using it well.	**46**
silent-corruption	The agent answers from false or stale content. Nobody gave it a new task. It simply believes a wrong thing.	**21**
persistence-backdoor	The bad content is written to memory or to a store. It fires on a later run, maybe for another user.	**14**
data-exfiltration	The agent sends out what it can read. It uses a tool call, a link or a message to move the data.	**13**
resource-abuse	No data moves and no goal changes. The agent burns time or refuses to answer at all.	**3**

The second row is the ML/AI track. The other five are mostly the security track. That split is the point of the paper.

Sources: goal-hijack Greshake et al., AISec@CCS 2023, arXiv:2302.12173 · reasoning-corruption Liu et al., TACL 2023, arXiv:2307.03172; Shi et al., ICML 2023, arXiv:2302.00093 · silent-corruption Xie et al., ICLR 2023 · persistence-backdoor Chen et al., NeurIPS 2024, arXiv:2407.12784 · data-exfiltration Qi et al., ICLR 2024 · resource-abuse Xue et al., 2024, arXiv:2406.00083

And where a defense can stand

INTENT: DID SOMEONE MEAN IT?

adversarial

A person placed the text to get this effect. This is the security track.

incidental

Nobody placed it. The window is long, stale or badly ordered. This is the ML/AI track.

both

The paper treats the case in both ways.

INTERVENTION POINT: WHERE THE DEFENSE ACTS

ingestion 155

It checks the text as it arrives, before the model reads it.

reasoning 205

It changes how the model uses text that is already in the window.

execution 74

It checks the action the agent wants to take, after the model decides.

Intent is the axis that splits the two literatures. The same failure gets two names and two sets of defenses.

A tool returns a page with a hidden order. A tool returns a page that is simply ten times too long. Both reach the window through the same channel. One has an author. One does not.

The counts on the right are defenses, not attacks. 100 more defenses carry no intervention point: they act after the event, such as forensics, or we never coded them.

Seven questions, then the experiments

BASE	**What we measure**	3 axes: channel, intent, consequence
RQ1	**Taxonomy**	60.5% of the cube is empty
RQ2	**Cross-citation**	11.7% / 9.1% between tracks
RQ3	**Pollution census**	223 named mechanisms
RQ4	**Defense census**	534 named defenses
RQ5	**Coverage matrix**	119 mechanisms never defended
RQ6	**Generalization**	9 reconstructed case studies
RQ7	**Open problems**	7 ranked, falsifiable
NEW	**Transfer matrix**	16 cells that no paper reports

We follow a standard review protocol, and we run it across two literatures

We did not invent the protocol. Four surveys in our own corpus use it, and we follow them.
They apply the PRISMA reporting guidelines, or the Kitchenham review guidelines. We apply the same steps to two literatures at once.

Search

We search two tracks at once. One is adversarial. One is incidental.
We found 25,922 records.

Screen

Fixed rules decide in or out. A program applies them to the title and abstract.
A person reads every unclear case. Next slide.

Extract

We read the full text of 1,097 papers. We code 32 fields for each paper.
Channel and consequence use closed lists.

Synthesize

We join the registries into the coverage matrix. It shows which defense met which attack. Then we run the code ourselves.

Sources: Protocol adopted from surveys in our own corpus: Woesle et al., IEEE Access 2026 (PRISMA) · Duarte et al., IEEE Access 2026 (PRISMA, dual screening) · Geng et al., Computers, Materials & Continua 2025 (Kitchenham guidelines) · Wang et al., 2026, arXiv:2602.10453 (SoK on a systematic review)

SCREENING

The rules that put a paper in or out

A PAPER GOES IN IF ALL THREE HOLD

1. It is peer reviewed, or it is an arXiv preprint from 2023 to 2026.
2. It is about LLM agents. The agent uses tools, or retrieval, or memory, or several steps.
3. It gives an attack, a defense, a benchmark, or a measurement. The subject is context contamination, or loss of quality as the context grows.

A PAPER GOES OUT IF ANY ONE HOLDS

1. It is only about jailbreaks. There is no agent and no tool.
2. It poisons training data only. We study the context at run time, not the training set.
3. The system is not an LLM.

A program applies these rules to the title and the abstract. It looks for words from fixed lists.

The agent list holds words such as agent, tool use, RAG, memory, multi-step, context window.

The threat list holds prompt injection, context poisoning, knowledge conflict, retrieval poisoning. A threat word counts only with an action word next to it, such as attack, poison, detect or mitigate.

A person reads every paper that the program cannot decide. The program never removes a paper on its own.

Result: 1,158 papers screened. 1,026 in. 132 out, each with a written reason.

What the numbers are computed over

1,026

papers included

1,158 rows screened; 132 excluded with cause

rescreening_log.md records each exclusion.
It gives a reason for each one.

In September 2026 we found a defect in the rules. The rules looked for the word "agent". The first papers in this field do not use it. They say "LLM-integrated application". The rules dropped them. The corpus went from 1,008 to 1,158.

Which combinations of channel, intent, and consequence have been studied at all?

60.5%

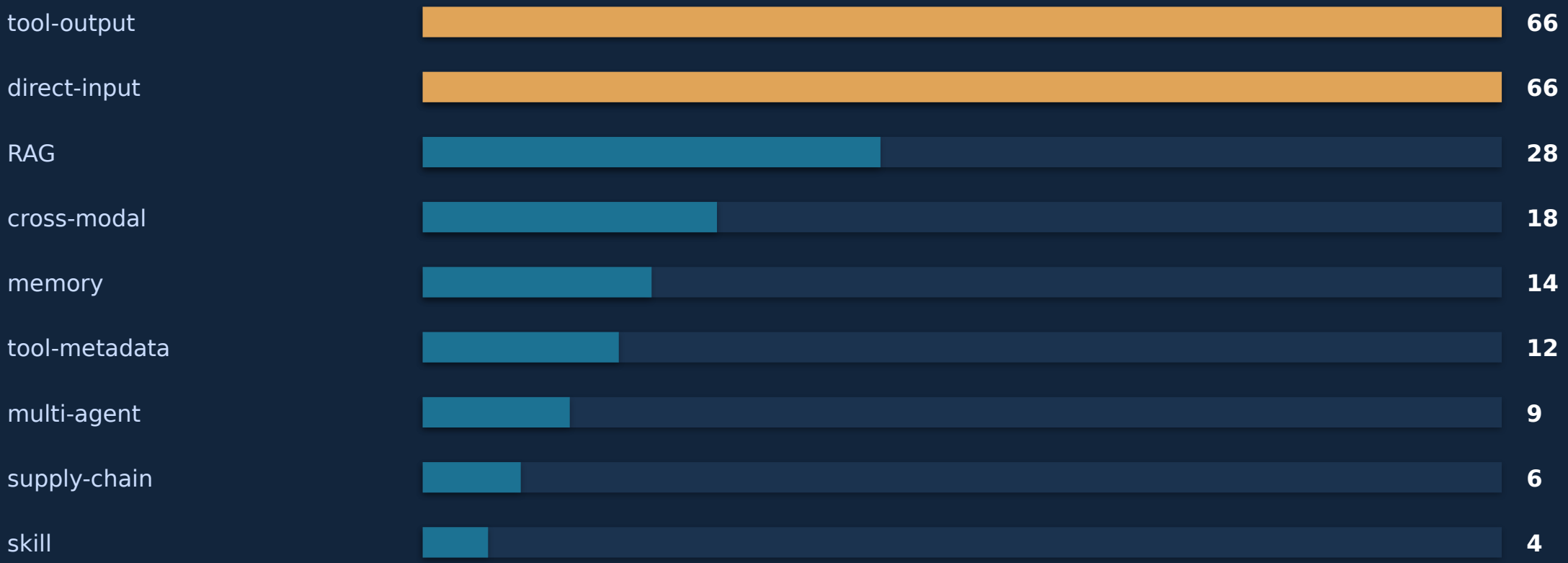
of the cube is empty

98 of 162 cells

9 channels × 3 intents × 6 consequences

Before the fix, 63.6% was empty.
The 150 new papers filled five cells.
They left 98 cells empty.

Where the work actually sits



Named mechanisms for each channel. direct-input and tool-output are equal.

RQ2 · CITATION NETWORK

Two literatures. Do they read each other?

Security → cites ML/AI

70 of 599 papers

11.7%

ML/AI → cites Security

37 of 407 papers

9.1%

Nine of ten papers do not cite the other track.

The corpus grew by 150 papers. The rates did not change. The split is structural.

How many distinct ways can a context be poisoned?

223

named mechanisms

168 adversarial · 55 incidental

Before the fix we had 183. We added 41.
40 of the new ones are adversarial.
23 of them use the direct-input channel.

This closed a channel gap that looked real.
Our own screening made that gap.
The field did not.

By consequence: goal-hijack 126 · reasoning-corruption 46 · silent-corruption 21 · persistence-backdoor 14 · data-exfiltration 13 · resource-abuse 3

Two numbers appear in this deck. 223 mechanisms carry a name. 14 more are known only by the benchmark that holds them. 237 in total.

And how many defenses answer them?

534

named defenses

2.8%

answer both threat models

Only 15 of 534 defenses answer both threat models: adversarial corruption and incidental corruption.

We added 55 defenses. These include StruQ, SecAlign, Spotlighting and the Instruction Hierarchy.
The number did not move.

2.9% → 2.8%.

Which defenses were ever tested against which attacks?

40%

of defenses have one test or more

119

mechanisms never defended

349 confirmed pairs

213 of 534 defenses matched

118 of 237 mechanisms covered

163 of them have one test only.

There are almost no general defenses.

Testing effort concentrates on a handful of targets



One mechanism takes 22.9% of all pairs. These six take 43.1%.

The other 119 mechanisms get no test at all.

We predicted the gap would close. It did not — and that is the result.

The prediction

We predicted this. The first defenses of the field will cover the mechanisms that look open. StruQ, SecAlign, Spotlighting, Attention Tracker and the Liu baselines will close the gap.

What happened

We added 164 papers. We added 55 defenses and 41 mechanisms. We got 45 new pairs.

Old mechanisms now covered:

zero.

The first 196 mechanisms did not move.

The new defenses and the new attacks cover each other. They cover nothing else.

A reviewer can say that our corpus limits make the gap. We tested that claim. It is not correct.

Does a defense built for one threat model hold against the other?

3

Full generalization

holds against the untested threat model

4

Partial / degraded

measurable protection, materially weaker

2

Inert

no measurable protection at all

We rebuilt 9 defenses from their released code or weights. We did not use the numbers in the papers.

The result follows the intervention point. Ingestion defenses usually hold. Execution defenses usually do not.

A confound applies. The ingestion harness is a text classifier. The execution harness is an agent. We state it.

The field has no standard benchmark to test on

42.6%

of attacks use data nobody else can reuse

11.7%

use the most common benchmark

255 distinct benchmarks
for 223 named attacks

199 of them (78%) are used by exactly one paper.

There are more benchmarks than attacks. Two papers can use one corpus and still not compare.

How long does a published defense survive?

96%

of published defeats land within one year

24 datable defeats, median gap 1 year, maximum 2.

Four teams defeated DataSentinel four separate times. A defense does not stop after one defeat.

Read this as an upper limit.
Only 2.81% of the registry has a recorded outcome. The sample prefers defenses that are well known. Attack papers go after those first.

TRANSFER

Detection depends on recognising the attack. Action-checking does not.

Attack it was built against

Three of four ingestion detectors
stop it completely.
The attack announces itself:
"an important message from Emma".

Same goal, different disguise

The same detectors stop working.
The payload forges a system turn,
so it reads as configuration,
not as an injected instruction.

The execution-stage technique blocked BOTH.
It never reads the payload — it asks whether the action serves the user's request,
which is invariant to how the instruction was dressed up.
RQ6 concluded execution defenses generalize least. This says the opposite.

Three rules. We fixed them before the runs.

RULE 1. THE CONTROL FIRES FIRST

A defense must first show its own benefit on its own task, with no attack present. If it cannot, we report no transfer result from it.

CK-PLUG fails this test on our data. Its two arms differ by 22.6 points before any attack runs. So we do not read its accuracy column at all.

RULE 2. THE CHANGE MUST BE LARGE ENOUGH TO SEE

Before a run we compute the smallest change that the sample size can detect. We use a two-proportion power test, alpha 0.05, power 0.80.

With 300 items that limit is 11.4 points. With 500 items it is 8.9 points.

A change below the limit is not a result. We write "not resolvable". We do not write "no effect".

RULE 3. WE USE THE AUTHORS' OWN SCORING CODE

We import the scoring function from each paper and run it without edits. We may change the harness around it. We never change how a result is scored.

We compared the scorers of two defenses line by line to confirm they match.

We run the defense code against attacks that no paper has tested it on

16

defense x attack
x model cells

94

measured
conditions

No paper reports these pairs.
That is why we chose them.

4 attacks. 5 defenses.
3 models. 2 defense families.

We change the attack dose from
0 to 10 bad passages of 10.
Each cell is a curve.

Three rules control every result.

The control must fire first. A change below the detection limit is not a result. We do not change the authors' scoring code.

Sources: PoisonedRAG Zou et al., USENIX Security 2024, arXiv:2402.07867 · BadRAG Xue et al., 2024, arXiv:2406.00083 · RobustRAG Xiang et al., 2024, arXiv:2405.15556 · ParamMute Huang et al., NeurIPS 2025, arXiv:2502.15543

RESULT 1

At full attack dose, one defense family fails and the other does not

Context-reliance

It holds down memory so the model follows the retrieved text.
ParamMute, CK-PLUG, SpARE

4 of 5 usable cells lose accuracy we can measure

Isolate-then-aggregate

It answers each passage alone, then joins the answers.
RobustRAG KeywordAgg, DecodingAgg

0 of 8 usable cells lose accuracy we can measure

The loss is the same size every time: -31.7 and -31.8 points, on two defenses, two attacks and two models.
Against a false passage the loss starts at 8 bad passages of 10. It does not wait for 10 of 10.

Design rule: do not hold down the model's own memory. It is the only thing left when the evidence goes.

Sources: ParamMute Huang et al., NeurIPS 2025, arXiv:2502.15543 · RobustRAG Xiang et al., 2024, arXiv:2405.15556 · PoisonedRAG Zou et al., USENIX Security 2024, arXiv:2402.07867 · BadRAG Xue et al., 2024, arXiv:2406.00083 · all numbers: our runs

RESULT 2

Some defenses make the attack work. Without them it stays near zero.

Defense	Attack	No defense	With defense	Dose
CK-PLUG	Tone steering	0.0 %	**78.2 %**	1 of 10
CK-PLUG	Tone steering	4.0 %	**99.0 %**	10 of 10
ParamMute	Tone steering	3.4 %	**28.8 %**	10 of 10
RobustRAG KeywordAgg, Llama-3	Refusal	0.0 %	**85.0 %**	9 of 10
RobustRAG KeywordAgg, Mistral	Refusal	0.0 %	**5.7 %**	9 of 10

With no defense the attack stays at or near zero at every dose. It works only through the defense.

Tone steering does not change the answer. A defense that pushes the model to follow the text follows the tone too.

The last row is the warning: the same defense helps on one model and does harm on another.

Sources: BadRAG Selective-Fact and DoS Xue et al., 2024, arXiv:2406.00083 · RobustRAG Xiang et al., 2024, arXiv:2405.15556 · ParamMute Huang et al., NeurIPS 2025, arXiv:2502.15543 · all numbers: our runs

RESULT 3

Two versions of one defense give opposite answers

RobustRAG ships both. They share the isolation step. They differ only in how they join the answers.

Bad passages of 10	1	5	6	7	8	9	10
KeywordAgg	+7.3	+4.0	+4.0	+5.7	+6.7	+12.3	-5.7
DecodingAgg	+8.7	-9.0	-14.0	-18.3	-20.3	-17.3	-4.0
Difference	**1.4**	**13.0**	**18.0**	**24.0**	**27.0**	**29.6**	**1.7**

At 9 bad passages of 10, one version helps and the other does harm. They are 29.6 points apart.

We wrote the cause down before we ran the test. A keyword filter throws away a passage with no answer in it, so one good passage still carries the answer. An average of the outputs lets nine bad ones drown it.

So the difference must grow as good passages go, and must vanish when none are left. It does both.

Sources: KeywordAgg and DecodingAgg both ship in Xiang et al., 2024, arXiv:2405.15556 · BadRAG DoS Xue et al., 2024, arXiv:2406.00083 · all numbers: our runs

DIFFERENTIATION

Against the five closest SoKs

closest competitor: The Landscape of Prompt Injection Threats (2602.10453)

papers reviewed	78	**1,026**
attacks catalogued	37	**223**
defenses catalogued	41	**534**
attack × defense pairs	—	**349**
runs own experiments	yes	**yes**
covers BOTH literatures	no	**YES**

They also run experiments. That is not the difference.

Four of the five closest surveys stay on one side of the divide. They do not mention it. The large ML survey does not cite security work. The closest security SoK does not cite degradation work.

Seven problems, ranked by expected value

1	**The tracks do not test each other**	2.0% of pairs cross the divide
2	**The uncovered tail**	119 mechanisms have no tested defense
3	**Single-mechanism validation is the norm**	163 defenses tested against exactly one
4	**Defenses meet only their contemporaries**	attack papers hold the later record
5	**Intervention point predicts generalization**	testable, and partly confounded today
6	**Benchmarks stand in for mechanisms**	one entry takes 22.9% of the pairs
7	**The problem is convention, not ability**	four of seven need no new science

SYNTHESIS

The field claims less coverage than it has. Some of what it has does harm.

Two literatures study the same problem. They do not read each other.

They name 223 ways to poison a context and 534 defenses. They almost never test one against the other.

The first limit is coordination. It is not ability.

RQ6 ran 7 defenses. 6 of them gave some protection against a threat model they never saw.

The protection is there. The test convention that would show it is not.

The gap is not ours. We added the literature we thought was missing, to close it. It did not close.

But coverage is not the whole result. When we ran the code, some defenses did not simply fail.

They made an attack work that stays near zero on a model with no defense at all.

Every number on these slides regenerates from the repo

config.yaml	screening criteria, applied by rule
rescreening_log.md	every corpus-affecting decision, dated, with cause
scripts/	dedupe → registries → coverage matrix → RQ1/RQ2/RQ7
context_sok_master_workbook.xlsx	papers, mechanisms, defenses, pairs, gaps
mcp_server/	the same data, queryable conversationally
data/paper_explorer.db	coverage and transfer figures, one source
scripts/build_presentation.py	this deck, built from the live data

We curate, define, score. We released code or weights. We never ask you to trust a paper we report about itself.

WHAT'S NEXT

From findings to manuscript

Finish the model control	run the second aggregation version on the other family's model; the first half shows the protection does not repeat
Add a third attack	AGGD, so the matrix does not rest on four attack mechanisms
Find an outside defense	the aggregation family comes from one repository; a second group's code would test the rule
Close the 99 near-misses	defense papers that name an attack in our registry but never test it
Manuscript draft	RQ1 to RQ7 are complete. The threats to validity are written.